

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

KHOA CÔNG NGHỆ THÔNG TIN



BÁO CÁO BÀI TẬP LỚN

Môn học: Lập trình với Python

Phân tích dữ liệu cầu thủ EPL 2024–2025

Giảng viên hướng dẫn: Kim Ngọc Bách

Nhóm thực hiện: 18

Các thành viên: Vũ Trung Đức - B23DCKH030

Ngô Văn Phương - B23DCKH086

Lê Việt Hoàng - B23DCKH044

Hà Nội – 2025

MỤC LỤC

Câu 1: Viết chương trình Python thu thập dữ liệu phân tích cầu thủ với yêu cầu như sau:	4
1.1. Thống kê các cầu thủ thi đấu nhiều hơn 90 phút ở Premier League mùa 2024-2025	4
1.1.1. Giới thiệu bài toán	4
1.1.2. Giải thích cách làm	4
1.1.2.1. Các thư viện cần thiết	4
1.1.2.2. Giải thích từng phần trong code	6
1.1.2.3 Tóm tắt luồng hoạt động (flow)	
1.1.3. Kết quả	11
1.2. Thống kê giá cầu thủ	
Với mỗi cầu thủ được ghi trong bảng trên, thu thập giá chuyển nhượng của cầu thủ trong mùa 2024-2025	11
1.2.1. Giới thiệu bài toán	11
1.2.2. Giải thích cách làm	12
1.2.2.1. Các thư viện cần thiết	12
1.2.3. kết quả	18
Câu 2:	18
2.1: Giới thiệu bài toán	18
2.1.1. Các thư viện đã sử dụng	19
2.1.2. Cấu hình đường dẫn database	19
2.1.3. Hàm tiện ích : query_db	20
2.1.4. Hàm get_player_by_name : API tra cứu cầu thủ	21
2.1.5 Hàm get_players_by_club(): API tra cứu theo club	22
2.1.6. Hàm index(): API mặc định	22
2.1.7. Hàm main()	23
Kết quả:	23
2.2. Giới thiệu bài toán	23
2.2.1. Các thư viện đã sử dụng	24
2.2.2. Cấu hình API và thư mục lưu kết quả	24
2.2.3. Hàm tra cứu cầu thủ theo tên	24
2.2.4. Hàm tra cứu cầu thủ theo câu lạc bộ	25
2.2.5. Hàm main()	26
Kết quả:	26
Câu 3:	27

3.1 Tính trung vị và độ lệch chuẩn của các chỉ số theo từng đội.....	27
3.1.1: Giới thiệu bài toán.....	27
3.1.2: Giải thích cách làm.....	27
3.1.2.1: Các thư viện cần thiết.....	27
3.1.2.2: Giải thích từng hàm.....	27
3.1.2.3: Giải thích luồng xử lý của chương trình.....	30
3.2 đề xuất 1 phương pháp định giá cầu thủ.....	32
3.2.1: Giới thiệu bài toán.....	32
3.2.2: Giải thích cách làm.....	32
3.2.2.1: Các thư viện cần dùng.....	32
3.2.2.2: Giải thích từng hàm.....	
3.2.2.3 Luồng xử lý của chương trình.....	37
3.2.3: Kết quả.....	39
Câu 4:.....	39
4.1. Bài toán 1.....	39
4.1.1. Các thư viện sử dụng.....	39
4.1.2. Xử lý số liệu.....	40
4.1.3. Sử dụng thuật toán K-Means để phân loại các cầu thủ thành các nhóm có chỉ số tương tự nhau.....	41
4.1.4. Vẽ biểu đồ Elbow và Sihoute.....	42
4.1.4.1. Elbow Method (Phương pháp Khuỷu tay).....	42
4.1.4.2. Silhouette Method (Phương pháp Silhouette).....	43
Kết luận:.....	43
Nhận xét: 4 nhóm được chia với các đặc điểm riêng khác biệt:.....	44
4.2. Bài toán 2.....	46
4.2.1. Thư viện cần thiết.....	46
4.2.1. Xác định k=4, huấn luyện kmeans.....	47
4.2.2. Tạo mô hình PCA, giảm chiều xuống còn 2, 3 và vẽ biểu đồ.....	47
4.2.2.1. Giảm số chiều dữ liệu xuống thành 2 và vẽ biểu đồ 2D.....	47
4.2.2.2. Giảm số chiều dữ liệu xuống thành 3 và vẽ biểu đồ 3D.....	48

Câu 1: Viết chương trình Python thu thập dữ liệu phân tích cầu thủ với yêu cầu như sau:

1.1. Thống kê các cầu thủ thi đấu nhiều hơn 90 phút ở Premier League mùa 2024-2025

1.1.1. Giới thiệu bài toán

- Thu thập dữ liệu thống kê của tất cả các cầu thủ có số phút thi đấu nhiều hơn 90 phút tại giải bóng đá ngoại hạng Anh mùa 2024-2025.
- Nguồn dữ liệu: <https://fbref.com/en/>
- Ghi kết quả vào một bảng trong cơ sở dữ liệu SQLite, bảng kết quả có cấu trúc như sau:

+ Mỗi cột tương ứng với một chỉ số.

+ Chỉ số nào không có hoặc không áp dụng thì để là N/a

+ Tham khảo:

<https://fbref.com/en/squads/822bd0ba/2023-2024//en/squads/822bd0ba/2023-2024/Liverpool-Stats>

1.1.2. Giải thích cách làm

1.1.2.1. Các thư viện cần thiết

```
import pandas as pd
from io import StringIO
import sqlite3
import os
import time
import random
from tqdm import tqdm
from bs4 import BeautifulSoup
import undetected_chromedriver as UC
from selenium.webdriver.chrome.options import Options
from selenium.common.exceptions import TimeoutException
```

pandas :Thư viện xử lý dữ liệu dạng bảng (DataFrame).

io.StringIO: Dùng để biến chuỗi HTML thành “file ảo”, vì pd.read_html mong muốn đối tượng giống file.

sqlite3: Thư viện chuẩn để làm việc với SQLite database.

os: Dùng để xử lý đường dẫn file, đảm bảo chạy đúng dù ở máy nào.

time random:

- + time.sleep() → tạo độ trễ giữa các request.
- + random.uniform(3, 6) → random thời gian sleep 3–6 giây → giả lập người dùng thật, giảm nguy

tqdm

- + Tạo thanh tiến trình đẹp khi lặp qua ALL_STATS_URL.
- + Hiện thị bạn đang crawl URL thứ mấy trên tổng số.

bs4 – BeautifulSoup

- + Dùng để phân tích (parse) mã HTML.
- + Nhiệm vụ trong bài:
 - Lấy driver.page_source (HTML sau khi đã render).
 - Dùng BeautifulSoup để tìm đúng <table> chứa dữ liệu cầu thủ.
 - Chọn bảng có id dạng stats_... và không có squads.

undetected_chromedriver (UC)

- + Đây là driver Selenium đã được tinh chỉnh để:
 - Khó bị nhận diện là bot hơn driver Selenium mặc định.
 - Phù hợp với các website có Cloudflare / anti-bot, như FBref.

selenium.webdriver.chrome.options.Options

- + Dùng để cấu hình trình duyệt Chrome:

selenium.common.exceptions.TimeoutException

- + Dùng để bắt lỗi timeout khi load trang:
 - Nếu bị timeout, chương trình retry tối đa 2 lần trước khi bỏ qua URL.

1.1.2.2. Giải thích từng phần trong code

1. Cấu hình đường dẫn & database

```
BASE_DIR = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))  
DATA_DIR = os.path.join(BASE_DIR, "data")  
DB_PATH = os.path.join(DATA_DIR, "stats.db")
```

- **BASE_DIR**: lên 3 cấp thư mục để trở về thư mục gốc project.
- **DATA_DIR**: thư mục chứa dữ liệu: .../data
- **DB_PATH**: đường dẫn đầy đủ đến file SQLite DB: .../data/stats.db

2. Danh sách endpoint FBref ([ALL_STATS_URL](#))

```
ALL_STATS_URL = [  
    "https://fbref.com/en/comps/9/2024-2025/stats/2024-2025-Premier-League-Stats",  
    "https://fbref.com/en/comps/9/2024-2025/keepers/2024-2025-Premier-League-Stats",  
    "https://fbref.com/en/comps/9/2024-2025/keepersadv/2024-2025-Premier-League-Stats",  
    "https://fbref.com/en/comps/9/2024-2025/shooting/2024-2025-Premier-League-Stats",  
    "https://fbref.com/en/comps/9/2024-2025/passing/2024-2025-Premier-League-Stats",  
    "https://fbref.com/en/comps/9/2024-2025/passing_types/2024-2025-Premier-League-Stats",  
    "https://fbref.com/en/comps/9/2024-2025/gca/2024-2025-Premier-League-Stats",  
    "https://fbref.com/en/comps/9/2024-2025/defense/2024-2025-Premier-League-Stats",  
    "https://fbref.com/en/comps/9/2024-2025/possession/2024-2025-Premier-League-Stats",  
    "https://fbref.com/en/comps/9/2024-2025/playingtime/2024-2025-Premier-League-Stats",  
    "https://fbref.com/en/comps/9/2024-2025/misc/2024-2025-Premier-League-Stats"  
]
```

- Mỗi URL là một loại thống kê khác nhau:
- Ý tưởng: crawl từng bảng, rồi gộp lại theo Player + Squad.

3. Hàm get_driver()

```
def get_driver():  
    opts = Options()  
    opts.page_load_strategy = "eager"  
    opts.add_argument("--no-sandbox")  
    opts.add_argument("--disable-gpu")  
    opts.add_argument("--disable-dev-shm-usage")  
    driver = UC.Chrome(version_main=141, options=opts)  
    driver.set_page_load_timeout(120)  
    return driver
```

Tạo một Chrome driver:

- `page_load_strategy = "eager"` → chỉ cần DOM load xong là đủ, không cần chờ ảnh → nhanh.
- Timeout 120 giây: tránh treo vĩnh viễn nếu mạng chậm.
- Có thể bật headless để chạy ngầm, không mở cửa sổ trình duyệt.

4. Hàm `setup_database()`

```
def setup_database():
    conn = sqlite3.connect(DB_PATH)
    cursor = conn.cursor()
    cursor.execute("DROP TABLE IF EXISTS PLAYER_STATS")
    conn.commit()
    conn.close()
    print(f"Database path: {DB_PATH}")
    print(f"Đã reset bảng PLAYER_STATS trong stats.db\n")
```

- Mục đích: reset bảng `PLAYER_STATS` mỗi lần chạy.
- Tránh chồng dữ liệu cũ – mới → đảm bảo DB luôn là dữ liệu mới nhất.

5. Hàm `save_to_database(df, table_name)`

```
def save_to_database(df, table_name):
    try:
        conn = sqlite3.connect(DB_PATH)
        df.to_sql(table_name, conn, if_exists="replace",
index=False)
        conn.commit()
        conn.close()
        print(f"Đã lưu vào bảng '{table_name}' trong stats.db!\n")
    except Exception as e:
        print(f"Lỗi khi lưu bảng {table_name}: {e}")
```

- Nhận DataFrame rồi lưu thẳng vào bảng SQL.
- `if_exists="replace"`: nếu bảng đã tồn tại → thay bằng dữ liệu mới.
- Dùng `try/except` để báo lỗi nếu lưu thất bại.

6. Hàm core: `get_player_stats()`

(1) Khởi tạo

```
df_list = []
driver = get_driver()
```

- df_list: dùng để lưu tạm các DataFrame nhỏ (mỗi URL một bảng).
- driver: 1 instance của trình duyệt dùng cho toàn bộ vòng lặp.

(2) Lặp qua từng URL trong ALL_STATS_URL

```
for url in tqdm(ALL_STATS_URL, total=len(ALL_STATS_URL),
desc="Đang thu thập stats"):
    success = False
    for attempt in range(2):
        try:
            driver.get(url)
            for _ in range(25):
                state = driver.execute_script("return
document.readyState;")
                if state == "complete":
                    break
            time.sleep(1)
            success = True
            break
        except TimeoutException:
            print(f"Timeout - thử lại {attempt+1}/2 ...")
            time.sleep(3)
        except Exception as e:
            print(f"Lỗi load URL: {url} - {e}")
            time.sleep(3)
    if not success:
        print(f"Bỏ qua URL: {url}\n")
        continue
```

Mỗi URL:

- Gọi driver.get(url) để mở trang.
- Dùng JS document.readyState để chờ trang load.
- Nếu lỗi TimeoutException → in cảnh báo, thử lại.
- Nếu lỗi khác → log ra, thử lại.

Nếu sau 2 lần vẫn lỗi → not success → bỏ qua URL đó.

(3) Chờ thêm & parse HTML bằng BeautifulSoup

```
time.sleep(random.uniform(3, 6))
soup = BeautifulSoup(driver.page_source, "html.parser")
stats_table = soup.find('table', id=lambda x: x and
x.startswith('stats_') and 'squads' not in x)
```

- sleep random 3–6 giây → giả lập user.

- page_source chứa HTML sau khi đã load JS.
- find('table', id=lambda x: x and x.startswith('stats_') and 'squads' not in x):
- Chọn bảng id kiểu stats_standard, stats_shooting, ... nhưng **loại bỏ** bảng mức đội (squads).

(4) Đọc bảng thành DataFrame

```
if stats_table is None:
    print(f"Không tìm thấy bảng ở URL: {url}")
    continue
df = pd.read_html(StringIO(str(stats_table)), header=1)[0]
df = df[df['Rk'] != 'Rk'].reset_index(drop=True)
if 'Rk' in df.columns:
    df = df.drop(columns=['Rk'])
df = df.set_index(['Player', 'Squad'])
df_list.append(df)
```

- read_html: đọc <table> thành DataFrame.
- FBref hay có dòng header lặp → df['Rk'] != 'Rk' để loại bỏ những dòng này.
- Loại cột Rk vì không cần thiết.
- Set index = (Player, Squad): Để khi concat theo axis=1, các bảng sẽ gộp theo đúng cầu thủ và đội bóng.
- Thêm DataFrame này vào df_list.

(5) Sau khi crawl xong tất cả URL

```
driver.quit()
if not df_list:
    print("Không có dữ liệu")
    return pd.DataFrame()
merge_df = pd.concat(df_list, axis=1)
merge_df = merge_df.loc[:, ~merge_df.columns.duplicated()]
merge_df = merge_df.reset_index()
```

- Đóng trình duyệt.
- pd.concat(df_list, axis=1): gộp theo chiều ngang (các cột), index là (Player, Squad).
- Xóa cột trùng lặp:
- + merge_df.columns.duplicated() là mask True/False
- + ~ để đảo ngược để giữ lại cột không trùng.
- Reset index để Player, Squad trở lại thành cột thường.

(6) Lọc cầu thủ có **Min > 90**

```
if "Min" in merge_df.columns:
```

```
merge_df["Min"] = pd.to_numeric(merge_df["Min"],
errors="coerce")
merge_df = merge_df[merge_df["Min"] > 90]
merge_df.fillna("N/A", inplace=True)
```

- Đảm bảo Min là kiểu số bằng to_numeric.
- Giữ lại cầu thủ có tổng số phút thi đấu > 90.
- Fill các giá trị thiếu bằng "N/A" để tránh NaN.

1.1.2.3 Tóm tắt luồng hoạt động (flow)

a. Khởi tạo

- Xác định đường dẫn thư mục & file database (BASE_DIR, DATA_DIR, DB_PATH).
- Khai báo danh sách URL thống kê FBref (ALL_STATS_URL).

b. Chuẩn bị database

- Hàm setup_database():
 - + Kết nối SQLite.
 - + Xóa bảng PLAYER_STATS nếu đã tồn tại.

c. Crawler

- Hàm get_player_stats():
 - + Tạo Chrome driver bằng undetected_chromedriver.
 - + Lặp từng URL trong ALL_STATS_URL:
 - Thử load trang tối đa 2 lần.
 - Chờ trang load (document.readyState == "complete").
 - Sleep ngẫu nhiên 3–6 giây để tránh bị chặn.
 - Dùng BeautifulSoup tìm bảng stats_*.
 - Đọc bảng HTML vào DataFrame bằng pandas.
 - Xử lý:
 - Xóa cột Rk, dòng header thừa.
 - Set index = (Player, Squad).
 - Thêm vào df_list.

d. Gộp và xử lý dữ liệu

- Sau khi crawl xong:
 - + Gộp tất cả DataFrame trong df_list theo chiều ngang (axis=1).
 - + Xóa cột trùng lặp.

- + Reset index để đưa Player, Squad về làm cột.
- + Ép kiểu cột Min sang số.
- + Lọc cầu thủ có Min > 90.
- + Fill các giá trị thiếu bằng "N/A".

e. Lưu kết quả

- Lưu DataFrame vào SQLite (PLAYER_STATS) bằng df.to_sql.
- Lưu DataFrame vào CSV (data/raw/player_stats.csv) bằng df.to_csv.

f. Kết thúc

- Đóng driver.
- In số lượng cầu thủ thu được & thông báo hoàn tất.

1.1.3. Kết quả

	Player	Squad	Nation	Pos	Age	Born	MP	Starts	Min	90s	Gls	Assists
1	Joshua Acheampong	Chelsea	eng ENG	DF	18	2006	4	2	170	1.9	0	0
2	Tyler Adams	Bournemouth	us USA	MF	25	1999	28	21	1965	21.8	0	3
3	Tosin Adarabioyo	Chelsea	eng ENG	DF	26	1997	22	15	1409	15.7	1	1
4	Simon Adingra	Brighton	ci CIV	FW,MF	22	2002	29	12	1097	12.2	2	2
5	Emmanuel Agbadou	Wolves	ci CIV	DF	27	1997	16	16	1410	15.7	1	0
6	Ola Aina	Nott'ham Forest	ng NGA	DF	27	1996	35	35	2995	33.3	2	1
7	Rayan Ait-Nouri	Wolves	dz ALG	DF	23	2001	37	37	3109	34.5	4	7
8	Kristoffer Ajer	Brentford	no NOR	DF	26	1998	24	17	1438	16.0	0	0
9	Manuel Akanji	Manchester City	ch SUI	DF	29	1995	26	23	2014	22.4	0	0
10	Nathan Aké	Manchester City	nl NED	DF	29	1995	10	8	678	7.5	0	0
11	Carlos Alcaraz	Everton	ar ARG	FW,MF	21	2002	15	7	764	8.5	2	3
12	Trent Alexander-Arnold	Liverpool	eng ENG	DF	25	1998	33	28	2365	26.3	3	6
13	Alisson	Liverpool	br BRA	GK	31	1992	28	28	2508	27.9	0	0
14	Miguel Almirón	Newcastle Utd	py PAR	FW	30	1994	9	1	157	1.7	0	0
15	Edson Álvarez	West Ham	mx MEX	MF	26	1997	28	20	1772	19.7	0	1
16	Harry Amass	Manchester Utd	eng ENG	DF	17	2007	5	4	346	3.8	0	0
17	Joachim Andersen	Fulham	dk DEN	DF	28	1996	29	29	2583	28.7	0	0
18	Elliot Anderson	Nott'ham Forest	eng ENG	MF	21	2002	37	33	2728	30.3	2	6
19	André	Wolves	br BRA	MF	23	2001	33	31	2472	27.5	0	0
20	Michail Antonio	West Ham	jm JAM	FW	34	1990	14	11	836	9.3	1	1
21	Antony	Manchester Utd	br BRA	DF,MF	24	2000	8	0	141	1.6	0	0
22	Julian Araujo	Bournemouth	mx MEX	DF	22	2001	12	7	497	5.5	0	0
23	Cameron Archer	Southampton	eng ENG	FW	23	2001	35	13	1441	16.0	2	0
24	Alphonse Areola	West Ham	fr FRA	GK	31	1993	26	25	2260	25.1	0	0
25	Joe Aribo	Southampton	ng NGA	MF,DF	28	1996	32	21	2013	22.4	3	0
26	Adam Armstrong	Southampton	eng ENG	FW,MF	27	1997	20	15	1248	13.9	2	2
27	Kepa Arrizabalaga	Bournemouth	es ESP	GK	29	1994	31	31	2790	31.0	0	0
28	Marco Asensio	Aston Villa	es ESP	MF	28	1996	13	9	743	8.3	3	1
29	Taiwo Awoniyi	Nott'ham Forest	ng NGA	FW	26	1997	26	3	416	4.6	1	0

1.2. Thống kê giá cầu thủ

Với mỗi cầu thủ được ghi trong bảng trên, thu thập giá chuyển nhượng của cầu thủ trong mùa 2024-2025

1.2.1. Giới thiệu bài toán

- Với mỗi cầu thủ được ghi trong bảng trên, thu thập giá chuyển nhượng của cầu thủ trong mùa 2024-2025 từ trang web <https://www.footballtransfers.com>. Nếu không có dữ liệu thì để là N/a

- Kết quả ghi vào một bảng thứ hai trong cơ sở dữ liệu SQLite.

1.2.2. Giải thích cách làm

1.2.2.1. Các thư viện cần thiết

- pandas: Dùng để xử lý dữ liệu dạng bảng (DataFrame), lọc dữ liệu và lưu CSV.
- sqlite3: Thư viện chuẩn của Python để kết nối, truy vấn và ghi dữ liệu vào SQLite database.
- os: Tạo và xử lý đường dẫn tới file database và thư mục dự án.
- selenium: Tự động điều khiển trình duyệt để crawl dữ liệu từ website.
- webdriver: Khởi tạo ChromeDriver để Selenium điều khiển trình duyệt.
- WebDriverWait / expected_conditions: Chờ phần tử xuất hiện trên trang (tránh lỗi tải chậm).
- By: Cách chọn phần tử HTML bằng CSS selector, ID, XPath,...
- ChromeDriverManager: Tự động tải phiên bản ChromeDriver phù hợp, tránh lỗi mismatch.
- selenium_stealth: “Ngụy trang” trình duyệt để tránh bị website phát hiện bot, giảm lỗi 403/429.
- time: Tạo độ trễ giữa các lần crawl, hạn chế rate limit.
- random: Thêm delay ngẫu nhiên giúp bot tự nhiên hơn.
- sys: Thoát chương trình nếu xảy ra lỗi nghiêm trọng (ví dụ không load được DB).

1.2.2.2. Giải thích cấu trúc chương trình

1) Hàm tạo bảng PLAYER_VALUES

```
def create_PLAYER_VALUES():
    conn = sqlite3.connect(DB_PATH)
    cur = conn.cursor()

    cur.execute("""
        CREATE TABLE IF NOT EXISTS "PLAYER_VALUES" (
            id INTEGER PRIMARY KEY AUTOINCREMENT,
            player_name TEXT,
            transfer_value TEXT
        )
    """)
    conn.commit()
    conn.close()
```

```
print("Table 'PLAYER_VALUES' checked/created  
successfully.")
```

Tạo bảng trong SQLite nếu chưa tồn tại.

- Bảng có 2 cột:
 - player_name
 - transfer_value

Đây là bảng chứa giá trị chuyển nhượng của từng cầu thủ.

2) Hàm xóa dữ liệu cũ

```
def clear_PLAYER_VALUES():  
    conn = sqlite3.connect(DB_PATH)  
    cur = conn.cursor()  
    cur.execute('DELETE FROM "PLAYER_VALUES"')  
    cur.execute("DELETE FROM sqlite_sequence WHERE  
name='PLAYER_VALUES'")  
    conn.commit()  
    conn.close()  
    print("Đã xóa dữ liệu cũ trong bảng 'PLAYER_VALUES' và  
reset ID.")
```

Xóa toàn bộ dữ liệu (DELETE)

- Đặt lại AUTO_INCREMENT (sqlite_sequence)

Mỗi lần chạy mới là dữ liệu mới hoàn toàn.

3) Hàm chèn dữ liệu (INSERT)

```
def insert_PLAYER_VALUES(df):  
    conn = sqlite3.connect(DB_PATH)  
    df.to_sql("PLAYER_VALUES", conn, if_exists="append",  
index=False)  
    conn.commit()  
    conn.close()  
    print(f"Đã lưu {len(df)} bản ghi vào bảng PLAYER_VALUES")
```

Dùng `df.to_sql()` để append dữ liệu mới vào DB.

4) Hàm khởi tạo ChromeDriver với Stealth

```
def get_driver():  
    service = Service(ChromeDriverManager().install())  
    options = webdriver.ChromeOptions()
```

```

options.add_argument("--disable-blink-features=AutomationContr
olled")
options.add_argument("window-size=1920,1080")
options.add_argument("--no-sandbox")
options.add_argument("--disable-dev-shm-usage")
driver = webdriver.Chrome(service=service, options=options)
stealth(
    driver,
    languages=["en-US", "en"],
    vendor="Google Inc.",
    platform="Win32",
    webgl_vendor="Intel Inc.",
    renderer="Intel Iris OpenGL Engine",
    fix_hairline=True
)
return driver

```

- Bao gồm:
 - Tải ChromeDriver tự động (webdriver_manager)
 - Tắt automation flags
 - Fake thông tin trình duyệt bằng selenium_stealth
- Cơ chế Stealth
 - Fake user-agent
 - Fake vendor
 - Fake platform
 - Fake WebGL rendering engine
 - Giảm bị phát hiện robot

Nếu không có Stealth → FootballTransfers sẽ chặn.

5) Hàm crawl giá chuyển nhượng của 1 cầu thủ

```

def get_transfer_value(driver, player_name: str):
    q = player_name.replace(" ", "%20")
    search_url =
f"https://www.footballtransfers.com/us/search?search_value={q}
"

    try:
        driver.get(search_url)

        el = WebDriverWait(driver, 8).until(

```

```

        EC.presence_of_element_located((By.CSS_SELECTOR,
"span.player-tag"))
    )

    return el.text.strip()

except Exception:
    return None

```

Luồng:

1. Truy cập trang search FootballTransfers
2. Chờ element span.player-tag xuất hiện
3. Lấy text → chính là giá trị chuyển nhượng
4. Trả về None nếu không tìm thấy

6) Hàm crawl toàn bộ danh sách

```

def scrape_value_for_player_list(players: list, delay: float =
1.0):
    driver = get_driver()
    collected = []
    try:
        total = len(players)
        for idx, name in enumerate(players, start=1):
            print(f"\n({idx}/{total}) Đang lấy: {name}")
            value = get_transfer_value(driver, name)
            print(f"    {name} → {value}")
            collected.append({
                "player_name": name,
                "transfer_value": value
            })
            # delay random nhẹ
            time.sleep(delay + random.uniform(0.2, 0.6))
        if collected:
            df = pd.DataFrame(collected)
            # SAVE DB
            insert_PLAYER_VALUES(df)
        else:
            print("Không có dữ liệu!")
    except Exception as e:
        print(f"Lỗi khi cào dữ liệu: {e}")
    finally:
        driver.quit()

```

```
print("Đã đóng trình duyệt.")
```

Luồng của hàm này:

1. Tạo driver Stealth
2. Duyệt từng tên cầu thủ
3. In tiến độ
4. Gọi `get_transfer_value()`
5. Lưu vào list collected
6. Dừng 1.2 – 1.6s để tránh rate-limit
7. Sau khi xong: Lưu vào database
8. Đóng trình duyệt

7) Hàm `main()` – nơi điều phối toàn bộ chương trình

```
if __name__ == "__main__":

    # 1) Tạo bảng
    create_PLAYER_VALUES()

    # 2) Xóa dữ liệu cũ
    clear_PLAYER_VALUES()

    # 3) Load from PLAYER_STATS
    try:
        print("\nĐang load danh sách từ bảng PLAYER_STATS...")

        conn = sqlite3.connect(DB_PATH)
        df_stats = pd.read_sql_query('SELECT Player FROM
PLAYER_STATS', conn)
        conn.close()

        df_stats.rename(columns={"Player": "player_name"},
inplace=True)
        player_list = df_stats["player_name"].dropna().tolist()

        print(f"Load thành công {len(player_list)} cầu thủ từ
bảng PLAYER_STATS")

    except Exception as e:
        print("\nKhông load được danh sách từ PLAYER_STATS.")
        print(e)
        sys.exit(1)
```

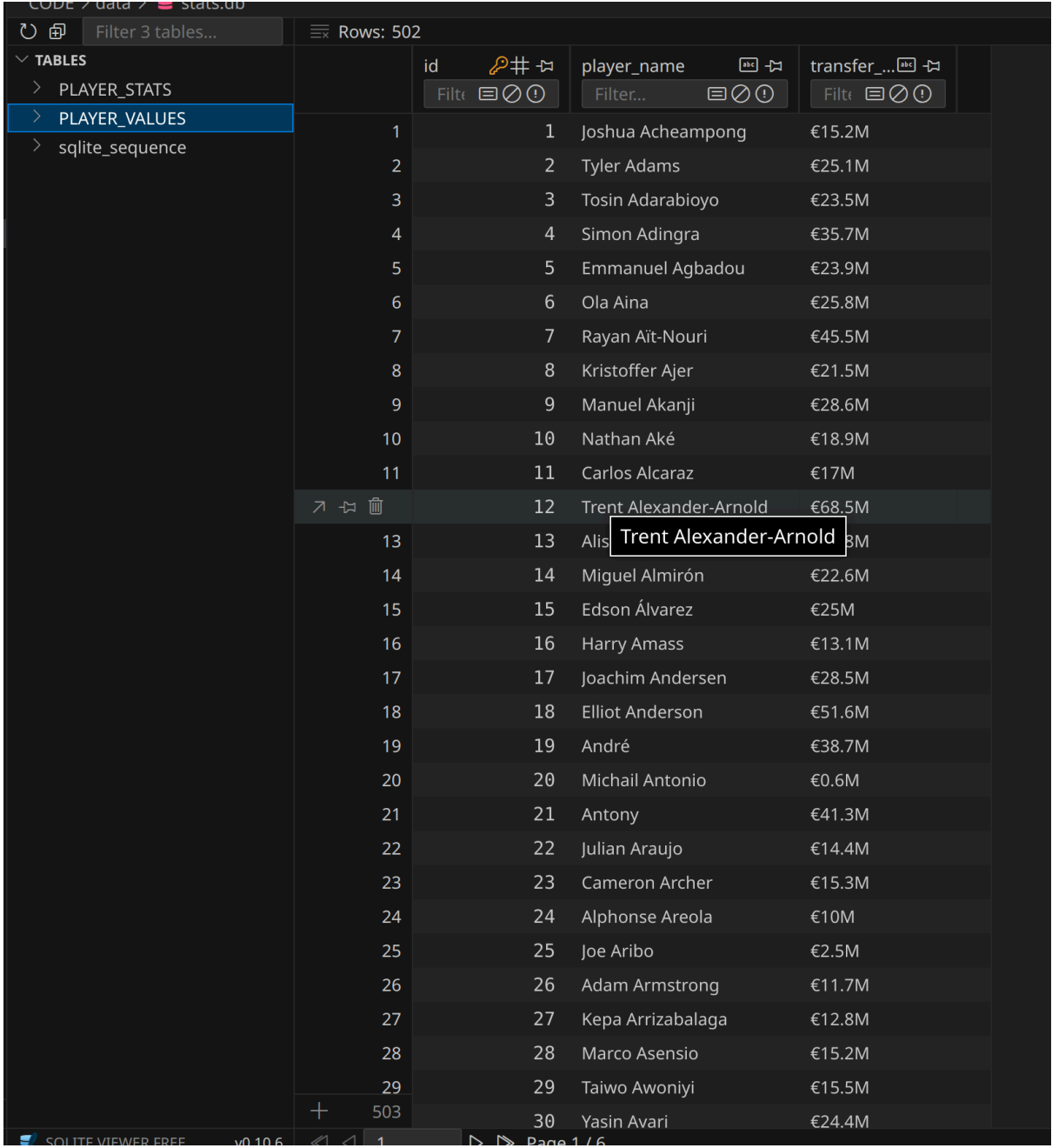


```
# 4) Crawl
print("\n---BẮT ĐẦU CRAWL---")
scrape_value_for_player_list(player_list)
```

Luồng:

1. Tạo bảng PLAYER_VALUES
2. Xóa dữ liệu cũ
3. Load danh sách cầu thủ từ bảng PLAYER_STATS
4. Gọi hàm crawl
5. Đóng chương trình

1.2.3. kết quả



SQLite Viewer - Filter 3 tables... Rows: 502

TABLES

- PLAYER_STATS
- PLAYER_VALUES
- sqlite_sequence

	id	player_name	transfer_...
1	1	Joshua Acheampong	€15.2M
2	2	Tyler Adams	€25.1M
3	3	Tosin Adarabioyo	€23.5M
4	4	Simon Adingra	€35.7M
5	5	Emmanuel Agbadou	€23.9M
6	6	Ola Aina	€25.8M
7	7	Rayan Aït-Nouri	€45.5M
8	8	Kristoffer Ajer	€21.5M
9	9	Manuel Akanji	€28.6M
10	10	Nathan Aké	€18.9M
11	11	Carlos Alcaraz	€17M
12	12	Trent Alexander-Arnold	€68.5M
13	13	Alis Trent Alexander-Arnold	€3M
14	14	Miguel Almirón	€22.6M
15	15	Edson Álvarez	€25M
16	16	Harry Amass	€13.1M
17	17	Joachim Andersen	€28.5M
18	18	Elliot Anderson	€51.6M
19	19	André	€38.7M
20	20	Michail Antonio	€0.6M
21	21	Antony	€41.3M
22	22	Julian Araujo	€14.4M
23	23	Cameron Archer	€15.3M
24	24	Alphonse Areola	€10M
25	25	Joe Aribo	€2.5M
26	26	Adam Armstrong	€11.7M
27	27	Kepa Arrizabalaga	€12.8M
28	28	Marco Asensio	€15.2M
29	29	Taiwo Awoniyi	€15.5M
30	30	Yasin Avari	€24.4M

SOLITE VIEWER FREE v0.10.6 Page 1 / 6

Câu 2:

2.1: Giới thiệu bài toán

Sử dụng module Python Flask để tạo các Restful API tra cứu chỉ số cầu thủ trong cơ sở dữ liệu vừa tạo ở câu 1 theo các tiêu chí sau:

- Tên cầu thủ: trả về toàn bộ chỉ số của cầu thủ có tên tương ứng.

- Câu lạc bộ: Trả về toàn bộ chỉ số của các cầu thủ thuộc về câu lạc bộ đó.

File gốc : [app.py](#)

2.1.1. Các thư viện đã sử dụng

```
from flask import Flask, request, jsonify
import sqlite3
import os
```

from flask import Flask, request, jsonify

- + Flask: tạo ứng dụng web/API backend.
- + request: lấy tham số từ URL
- + jsonify: chuyển dữ liệu Python → JSON để trả về cho client.

import sqlite3 : Dùng để kết nối cơ sở dữ liệu SQLite.

import os: Đảm bảo chương trình chạy đúng thư mục bất kể bạn chạy từ đâu

2.1.2. Cấu hình đường dẫn database

```
BASE_DIR = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
DB_PATH = os.path.join(BASE_DIR, "data", "stats.db")
app = Flask(__name__)
```

- BASE_DIR : lùi 3 cấp để đưa về đúng đường dẫn :
BTL-Python/src/api/[app.py](#)
- DB_PATH : ghép đường dẫn với /data/[stats.db](#)
- app = Flask(__name__) : tạo ứng dụng Flask

2.1.3. Hàm tiện ích : query_db

```
def query_db(query, params=()):  
  
    conn = sqlite3.connect(DB_PATH)  
  
    conn.row_factory = sqlite3.Row  
  
    cur = conn.cursor()  
  
    cur.execute(query, params)  
  
    rows = cur.fetchall()  
  
    conn.close()  
  
    return [dict(row) for row in rows]
```

Đây là hàm trung gian giúp:

- + Mỗi API không phải lặp lại code kết nối DB.
- + Kết quả trả về ở format thân thiện, dễ jsonify.

Cụ thể :

- `conn = sqlite3.connect(DB_PATH)`: Mở kết nối tới file stats.db.
- `conn.row_factory = sqlite3.Row`
→ Quan trọng: thay vì trả về tuple, mỗi dòng sẽ là một object kiểu Row có thể lấy theo tên cột.
Ví dụ: `row["Player"]`.
- `cur = conn.cursor()`: Lấy cursor để thực hiện lệnh SQL.
- `cur.execute(query, params)`: Thực thi câu SQL.
 - + `query`: chuỗi SQL, ví dụ `SELECT * FROM ... WHERE Player LIKE ?`
 - + `params`: tuple chứa giá trị cho ? (dùng parameterized query để tránh SQL injection).

`rows = cur.fetchall()`: Lấy tất cả các dòng kết quả.

conn.close(): Đóng kết nối, giải phóng tài nguyên.

return [dict(row) for row in rows]: Mỗi row là sqlite3.Row, chuyển sang dict.

Ví dụ: sqlite3.Row → {"Player": "Bukayo Saka", "Gls": 16, ...} → Trả về list các dict, rất tiện cho jsonify().

2.1.4. Hàm get_player_by_name : API tra cứu cầu thủ

```
@app.route("/player", methods=["GET"])

def get_player_by_name():

    name = request.args.get("name", "").strip()

    if not name:

        return jsonify({"error": "Thiếu tham số ?name=<tên cầu thủ>"}), 400

    query = "SELECT * FROM PLAYER_STATS WHERE Player LIKE ? COLLATE NOCASE"

    results = query_db(query, (f"%{name}%",))

    if not results:

        return jsonify([]), 200

    return jsonify(results)
```

Trước khi vào hàm, sử dụng phương thức GET HTTP tới endpoint /player

=> Để Flask gọi hàm get_player_by_name()

- Hàm get_player_by_name():

- + Lấy tham số name từ URL . Vd: /player?name=Haaland: Nếu không thấy thì API trả về json như trên với trạng thái 400
- + Truy vấn DB, bỏ qua dấu và viết hoa thường: Nếu không thấy thì trả về rỗng, nếu có thì trả về list các dict

2.1.5 Hàm get_players_by_club(): API tra cứu theo club

```
@app.route("/club", methods=["GET"])
def get_players_by_club():
    club = request.args.get("club", "").strip()
    if not club:
        return jsonify({"error": "Thiếu tham số ?club=<tên câu lạc bộ>"}), 400
    query = "SELECT * FROM PLAYER_STATS WHERE Squad LIKE ? COLLATE NOCASE"
    results = query_db(query, (f"%{club}%",))

    if not results:
        return jsonify([]), 200

    return jsonify(results)
```

Tương tự hàm get_player_by_name()

- Lấy tham số club từ URL. Vd: /club?club=Arsenal
- Truy vấn DB

2.1.6. Hàm index(): API mặc định

```
@app.route("/", methods=["GET"])
def index():
    return jsonify({
        "message": "API tra cứu cầu thủ FBref + FootballTransfers",
        "endpoints": {
            "/player?name=<tên>": "Tra cứu theo tên cầu thủ",
            "/club?club=<club>": "Tra cứu theo tên câu lạc bộ"
        }
    })
```

- Khi truy cập <http://127.0.0.1:5000/> thì server trả:

- + Một JSON nhỏ mô tả API.
- + Tên các endpoint và cách dùng.

2.1.7. Hàm main()

```
if __name__ == "__main__":

    print(f"📡 API đang chạy tại: http://127.0.0.1:5000")

    app.run(debug=True)
```

- Khởi động Flask ở server 127.0.0.1:5000
- Tự động reload khi sửa code và có lỗi thì hiện debug

Kết quả:

```
(venv) C:\Users\admin\OneDrive\Attachments\Documents\GitHub\BTL-Python>python -m src.api.app
📡 API đang chạy tại: http://127.0.0.1:5000
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
📡 API đang chạy tại: http://127.0.0.1:5000
* Debugger is active!
* Debugger PIN: 128-354-105
```

2.2. Giới thiệu bài toán

Sử dụng module Request, viết 1 chương trình python để tra cứu dữ liệu cầu thủ theo 2 tiêu chí như trên. Tên cầu thủ hoặc câu lạc bộ được truyền vào lúc chạy chương trình có cú pháp như sau:

```
./> python lookup.py --name <tên cầu thủ> --club <tên câu lạc bộ>
```

- Kết quả trả ra được in ra màn hình dưới dạng bảng, và 1 file CSV. Tên file được đặt theo tên cầu thủ hoặc câu lạc bộ, tùy theo sử dụng tham số đầu vào.

File gốc : lookup.py

2.2.1. Các thư viện đã sử dụng

```
import requests

import argparse

import pandas as pd

import os
```

requests : Thư viện gửi HTTP request (thư viện chính).

argparse : Giúp nhận tham số từ dòng lệnh

- python -m [lookup.py](#) --name Haaland
- python -m [lookup.py](#) --club Manchester City

pandas : Xử lý dữ liệu dạng bảng

os : Thao tác đường dẫn ở mọi máy, hệ điều hành

2.2.2. Cấu hình API và thư mục lưu kết quả

```
API_URL = "http://127.0.0.1:5000"
BASE_DIR =
os.path.dirname(os.path.dirname(os.path.dirname(os.path.abspath(__file__))))
RESULT_DIR = os.path.join(BASE_DIR, "data", "lookup_results")
os.makedirs(RESULT_DIR, exist_ok=True)
```

API_URL : địa chỉ server Flask đang chạy (localhost)

BASE_DIR : lùi về sao cho đúng đường dẫn tuyệt đối

RESULT_DIR : nơi lưu kết quả **data/lookup_results/**

=> Nếu thư mục chưa có thì tạo mới

2.2.3. Hàm tra cứu cầu thủ theo tên

```
def lookup_player(name):
    print(f"Tra cứu cầu thủ: {name}")
    url = f"{API_URL}/player?name={name}"
    r = requests.get(url)
    if r.status_code != 200:
        print(f"Lỗi HTTP {r.status_code}: {r.text}")
    return
```



```

data = r.json()
if not data:
    print(f"Không tìm thấy cầu thủ {name}")
    return
df = pd.DataFrame(data)
output = os.path.join(RESULT_DIR, f"{name}_lookup.csv")
df.to_csv(output, index=False, encoding="utf-8-sig")
print(df.head(10))
print(f"Kết quả đã lưu: {output}")

```

- Đầu tiên là gửi request GET tới URL
- Nhận dữ liệu từ JSON rồi chuyển sang dạng Dataframe
- Lợi ích của DataFrame:
 - + In bảng đẹp
 - + Dễ xuất CSV
- Xuất ra file csv file đầy đủ sau khi tra. VD: Haaland_lookup.csv
- Hiện thị ra top 10

2.2.4. Hàm tra cứu cầu thủ theo câu lạc bộ

```

def lookup_club(club):
    print(f"Tra cứu câu lạc bộ: {club}")
    url = f"{API_URL}/club?club={club}"
    r = requests.get(url)
    if r.status_code != 200:
        print(f"Lỗi HTTP {r.status_code}: {r.text}")
        return
    data = r.json()
    if not data:
        print(f"Không tìm thấy CLB {club}")
        return
    df = pd.DataFrame(data)
    output = os.path.join(RESULT_DIR, f"{club.replace(' ', '_')}_lookup.csv")
    df.to_csv(output, index=False, encoding="utf-8-sig")
    print(df.head(10))
    print(f"Kết quả đã lưu: {output}")

```

Tương tự như tra cầu thủ theo tên

- Đầu tiên là gửi request GET tới URL
- Nhận dữ liệu từ JSON rồi chuyển sang dạng Dataframe

- Lợi ích của DataFrame:
 - + In bảng đẹp
 - + Dễ xuất CSV
- Xuất ra file csv file đầy đủ sau khi tra. VD: Manchester_City_lookup.csv
- Hiện thị ra top 10

2.2.5. Hàm main()

```
if __name__ == "__main__":
    parser = argparse.ArgumentParser(description="Tra cứu dữ liệu cầu thủ hoặc CLB từ API")
    parser.add_argument("--name", help="Tên cầu thủ")
    parser.add_argument("--club", help="Tên câu lạc bộ")
    args = parser.parse_args()

    if args.name:
        lookup_player(args.name)
    elif args.club:
        lookup_club(args.club)
    else:
        print("⚠️ Hãy truyền thêm --name hoặc --club")
```

Hỗ trợ và cấu hình dòng nhập lệnh theo yêu cầu sao cho chạy đúng hàm.

Giải thích:

- Nếu người dùng gõ --name, chạy lookup_player()
- Nếu gõ --club, chạy lookup_club()
- Nếu không truyền gì → báo lỗi hướng dẫn cách dùng

Kết quả:

- Tra cứu theo tên :
- Câu lệnh: `python -m CODE.src.api.lookup --name "Haaland"`

```
(.venv) taitu@taitu-Inspiron-14-5430:~/Documents/GitHub/BTL-Python$ python -m CODE.src.api.lookup --club "Manchester City"
Tra cứu câu lạc bộ: Manchester City
#OPA #OPA/90 +/- +/-90 /90 1/3 2CrDY 90s A-xAG Age Ast ... onx6 onxGA unSub xA xAG xAG.1 xG xG+/- xG+/-90 xG+xAG xG.1
0 N/A N/A +18 +0.80 N/A 177 0 22.4 -0.4 29 0 ... 43.7 29.1 2 1.3 0.4 0.02 1.6 +14.6 +0.65 0.09 0.07
1 N/A N/A 0 0.00 N/A 42 0 7.5 0.0 29 0 ... 12.6 13.5 9 0.2 0.0 0.00 0.2 -0.9 -0.12 0.03 0.03
2 N/A N/A +23 +1.22 N/A 88 0 18.9 -1.0 33 7 ... 34.4 21.1 5 6.7 8.0 0.42 5.1 +13.2 +0.70 0.69 0.27
3 N/A N/A +29 +1.15 N/A 222 0 25.2 -0.3 27 0 ... 46.2 28.8 0 1.2 0.3 0.01 1.5 +17.5 +0.69 0.07 0.06
4 N/A N/A +20 +1.19 N/A 12 0 16.8 +1.2 22 6 ... 33.5 14.4 4 3.8 4.8 0.29 1.3 +19.1 +1.13 0.36 0.08
5 49 1.90 +23 +0.89 +0.19 21 0 25.8 +2.5 30 4 ... 49.0 28.7 4 0.2 1.5 0.06 0.0 +20.3 +0.79 0.06 0.00
6 N/A N/A +6 +0.30 N/A 41 0 19.7 -2.8 24 2 ... 31.2 28.8 5 5.3 4.8 0.24 5.2 +2.4 +0.12 0.50 0.26
7 N/A N/A +10 +1.18 N/A 52 0 8.4 -0.1 22 0 ... 16.5 8.5 3 0.5 0.1 0.01 0.5 +8.0 +0.94 0.06 0.06
8 N/A N/A +7 +0.87 N/A 23 0 8.0 -0.8 28 1 ... 14.8 11.2 12 1.7 1.8 0.23 1.1 +3.6 +0.45 0.37 0.14
9 N/A N/A +25 +1.01 N/A 108 0 24.7 +2.5 33 6 ... 44.4 35.0 4 4.4 3.5 0.14 3.2 +9.5 +0.38 0.27 0.13

[10 rows x 176 columns]
Kết quả đã lưu: data/lookup_results/Manchester_City_lookup.csv
```

- Tra cứu theo club:
- Câu lệnh: `python -m CODE.src.api.lookup --club "Manchester City"`

```
(.venv) taitu@taitu-Inspiron-14-5430:~/Documents/GitHub/BTL-Python$ python -m CODE.src.api.lookup --club "Manchester City"
Tra cứu câu lạc bộ: Manchester City
#OPA #OPA/90 +/- +/-90 /90 1/3 2CrDY 90s A-xAG Age Ast ... onxG onxGA unSub xA xAG xAG.1 xG xG+/- xG+/-90 xG+xAG xG.1
0 N/A N/A +18 +0.80 N/A 177 0 22.4 -0.4 29 0 ... 43.7 29.1 2 1.3 0.4 0.02 1.6 +14.6 +0.65 0.09 0.07
1 N/A N/A 0 0.00 N/A 42 0 7.5 0.0 29 0 ... 12.6 13.5 9 0.2 0.0 0.00 0.2 -0.9 -0.12 0.03 0.03
2 N/A N/A +23 +1.22 N/A 88 0 18.9 -1.0 33 7 ... 34.4 21.1 5 6.7 8.0 0.42 5.1 +13.2 +0.70 0.69 0.27
3 N/A N/A +29 +1.15 N/A 222 0 25.2 -0.3 27 0 ... 46.2 28.8 0 1.2 0.3 0.01 1.5 +17.5 +0.69 0.07 0.06
4 N/A N/A +20 +1.19 N/A 12 0 16.8 +1.2 22 6 ... 33.5 14.4 4 3.8 4.8 0.29 1.3 +19.1 +1.13 0.36 0.08
5 49 1.90 +23 +0.89 +0.19 21 0 25.8 +2.5 30 4 ... 49.0 28.7 4 0.2 1.5 0.06 0.0 +20.3 +0.79 0.06 0.00
6 N/A N/A +6 +0.30 N/A 41 0 19.7 -2.8 24 2 ... 31.2 28.8 5 5.3 4.8 0.24 5.2 +2.4 +0.12 0.50 0.26
7 N/A N/A +10 +1.18 N/A 52 0 8.4 -0.1 22 0 ... 16.5 8.5 3 0.5 0.1 0.01 0.5 +8.0 +0.94 0.06 0.06
8 N/A N/A +7 +0.87 N/A 23 0 8.0 -0.8 28 1 ... 14.8 11.2 12 1.7 1.8 0.23 1.1 +3.6 +0.45 0.37 0.14
9 N/A N/A +25 +1.01 N/A 108 0 24.7 +2.5 33 6 ... 44.4 35.0 4 4.4 3.5 0.14 3.2 +9.5 +0.38 0.27 0.13

[10 rows x 176 columns]
Kết quả đã lưu: data/lookup_results/Manchester_City_lookup.csv
```

Câu 3:

3.1 Tính trung vị và độ lệch chuẩn của các chỉ số theo từng đội

3.1.1: Giới thiệu bài toán

- Tìm trung vị, trung bình và độ lệch chuẩn của mỗi chỉ số các cầu thủ của mỗi đội. Ghi kết quả vào 1 file CSV.
- Tìm đội bóng có chỉ số điểm số cao nhất ở mỗi chỉ số. Theo bạn đội nào có phong độ tốt nhất giải ngoại Hạng Anh mùa 2024-2025

3.1.2: Giải thích cách làm

3.1.2.1: Các thư viện cần thiết

sqlite3: Thư viện chuẩn của Python dùng để kết nối vào SQLite database.

pandas: Dùng để xử lý dữ liệu dạng bảng (DataFrame).

os: Tạo đường dẫn tới file database

3.1.2.2: Giải thích từng hàm

1. `create_connection()` – Tạo kết nối tới SQLite

```
def create_connection():
    os.makedirs(DATA_DIR, exist_ok=True)
    return sqlite3.connect(DB_PATH)
```

- Đảm bảo thư mục data/ tồn tại.
- Mở kết nối tới file stats.db.
- Nếu file chưa tồn tại → SQLite sẽ tự tạo file.

2. analyze_stats() – HÀM CHÍNH XỬ LÝ TOÀN BỘ

Bước 1 – Kết nối database và kiểm tra bảng

```
conn = create_connection()
print("DB_PATH:", DB_PATH)
print("Exists:", os.path.exists(DB_PATH))
tables = pd.read_sql_query(
    "SELECT name FROM sqlite_master WHERE type='table';",
    conn
)
print("Các bảng trong DB:", tables["name"].tolist())
if "PLAYER_STATS" not in tables["name"].values:
    raise Exception("Không tìm thấy bảng 'PLAYER_STATS' trong database!")
```

Lấy danh sách tất cả bảng trong DB.

Kiểm tra bảng PLAYER_STATS đã tồn tại hay chưa.

+ Nếu chưa có → báo lỗi vì không có dữ liệu để phân tích.

Bước 2 – Load dữ liệu từ bảng PLAYER_STATS

```
df = pd.read_sql_query("SELECT * FROM PLAYER_STATS;", conn)
conn.close()
print("Loaded stats:", df.shape)
print("Columns:", df.columns.tolist())
```

Tải toàn bộ bảng vào DataFrame.

In kích thước (df.shape) và danh sách cột.

Bước 3 – Loại bỏ cột không cần thiết

```
drop_cols = [
    "Player", "Squad", "Pos", "Nation", "Comp", "Born",
    "Age", "Matches", "Games", "MP", "Starts", "Min"
]
df_numeric = df.drop(columns=[c for c in drop_cols if c in df.columns], errors="ignore")
```

Các cột này bị drop để đảm bảo chỉ phân tích **chỉ số thuần túy**.

Bước 4 – Ép kiểu các cột thành số nếu có thể

```
df = df.apply(pd.to_numeric, errors="ignore")
```

Các cột dạng ký tự → giữ nguyên

Các cột dạng số → convert thành float hoặc int

Cột không convert được → bỏ qua

Bước 5 – Xác định các cột dạng số để phân tích

```
numeric_cols =
df.select_dtypes(include="number").columns.tolist()
```

```
print("\nNumeric columns:", len(numeric_cols))
```

- Lấy tất cả cột thuộc kiểu: int và float

Bước 5 – Thay NA bằng 0

```
df[numeric_cols] = df[numeric_cols].fillna(0)
```

Cực kỳ quan trọng vì:

- + median/mean/std cần dữ liệu numeric
- + Nếu NA nhiều → thống kê sai lệch

Bước 6 – Tính median, mean, std theo từng đội

```
stats = df.groupby("Squad")[numeric_cols].agg(["median",  
"mean", "std"])
```

Đây là bảng thống kê chuẩn phục vụ phân tích chiến thuật.

Bước 7 – Flatten MultiIndex để lưu CSV

```
stats.columns = ["_".join(col) for col in stats.columns]  
  
stats_file = os.path.join(DATA_DIR, "team_summary_stats.csv")  
stats.to_csv(stats_file)  
print(f"Đã tạo file {stats_file}")
```

Xuất ra file csv

Bước 8: Tìm đội mạnh nhất theo từng chỉ số

```
mean_cols = [c for c in stats.columns if c.endswith("_mean")]  
negative_patterns = [  
    "foul", "crdy", "crdr", "2crdy",  
    "dispos", "mis", "error",  
    "pkcon", "off",  
    "tkl_dribbled", "dribbled_past"  
]  
  
positive_mean_cols = [  
    c for c in mean_cols  
    if not any(neg in c.lower() for neg in negative_patterns)  
]  
  
print(f"Số metric tích cực dùng để tìm best team:  
{len(positive_mean_cols)}")  
  
best_by_metric = stats[positive_mean_cols].idxmax()  
best_by_metric.index = [c.replace("_mean", "") for c in  
best_by_metric.index]  
best_by_metric.name = "BestTeam"  
best_by_metric.index.name = "Metric"  
best_metric_file = os.path.join(DATA_DIR,  
"best_team_each_metric.csv")
```

```
best_by_metric.to_csv(best_metric_file)
print(f"Đã tạo file đội mạnh nhất từng chỉ số (positive metrics only): {best_metric_file}")
print("\nĐội mạnh nhất theo từng metric:")
print(best_by_metric)
```

- + Chỉ chọn mean của từng metric
- + Loại bỏ các metric tiêu cực (fouls, errors, displaced...)
- + Tìm đội mạnh nhất cho từng metric tích cực
- + Làm sạch dữ liệu trước khi ghi CSV
- + Xuất bảng best team theo metric

Bước 9: Tìm đội có phong độ tổng thể tốt nhất

```
team_overall_score =
stats[positive_mean_cols].mean(axis=1).round(2)

best_team_overall = team_overall_score.idxmax()

print(f"\nĐội có phong độ tổng thể cao nhất EPL 2024-2025:")

print(f"{best_team_overall} ")

return stats, best_by_metric
```

- + Chương trình tính điểm phong độ tổng hợp của mỗi đội dựa trên các chỉ số thi đấu mang tính tích cực như xG, xA, số cú sút, đường chuyền thành công, tắc bóng, tạo cơ hội hay gây áp lực. Các chỉ số tiêu cực như phạm lỗi, thẻ phạt, mất bóng, bị qua người hay gây phạt đền được loại bỏ để tránh làm sai lệch đánh giá.
- + Sau khi tính trung bình toàn bộ các chỉ số tích cực, đội có giá trị trung bình cao nhất được xem là đội có phong độ tổng thể tốt nhất EPL 2024–2025.

3.1.2.3: Giải thích luồng xử lý của chương trình

Bước 1 – Kết nối và đọc dữ liệu

- Chương trình tạo kết nối tới SQLite (stats.db).
- Đọc toàn bộ dữ liệu bảng PLAYER_STATS vào DataFrame Pandas.

Bước 2 – Làm sạch dữ liệu

- Loại bỏ các cột không cần thiết như thông tin cá nhân (quốc tịch, độ tuổi, số trận,...).
- Chuyển toàn bộ các cột còn lại sang dạng số (numeric).
- Điền NA bằng 0 để tránh lỗi khi tính toán.
- Gắn lại cột Squad để phục vụ phân nhóm theo đội.

Bước 3 – Tính toán thống kê theo đội

- Nhóm dữ liệu theo Squad (tên đội bóng).
- Tính median, mean, std cho tất cả các chỉ số của cầu thủ thuộc mỗi đội.
- Làm phẳng tên cột và làm tròn 2 chữ số thập phân.
- Lưu kết quả vào file `team_summary_stats.csv`.

Bước 4 – Lọc chỉ số tích cực (Positive Metrics)

- Lấy danh sách tất cả các cột trung bình (*_mean).
- Loại bỏ các chỉ số tiêu cực như fouls, thẻ phạt, lỗi, mất bóng, bị vượt qua người, việt vị,... chỉ giữ lại những chỉ số phản ánh hiệu suất tốt của cầu thủ.

Bước 5 – Tìm đội mạnh nhất theo từng chỉ số

- Với mỗi chỉ số tích cực, tìm đội có giá trị mean cao nhất.
- Kết quả: đội mạnh nhất ở từng metric.
- Lưu vào file `best_team_each_metric.csv`.

Bước 6 – Tính đội có phong độ tổng thể cao nhất

- Lấy trung bình toàn bộ các positive metrics của từng đội.
- Đội có giá trị trung bình cao nhất được xem là đội có phong độ tổng thể tốt nhất EPL.
- Chỉ in ra màn hình, không lưu file.

Bước 7 – Trả về kết quả

- Trả về bảng thống kê và bảng đội mạnh nhất theo từng metric

3.1.3 Kết quả

	Squad	90s_median	90s_mean	90s_std	Gls_median	Gls_mean	Gls_std	Ast_median	Ast_mean	Ast_std	G+A_median	G+A_mean	G+A_std
2	Arsenal	18.4	18.88	10.46	2.5	3.05	2.9	2.0	2.5	2.94	3.5	5.55	5.13
3	Aston Villa	12.55	14.88	11.34	1.0	2.0	3.42	0.0	1.61	2.7	2.5	3.61	5.56
4	Bournemouth	19.6	17.31	12.36	1.0	2.38	3.7	1.0	1.71	2.01	2.0	4.08	5.23
5	Brentford	18.7	18.91	13.76	0.0	2.95	5.91	1.5	2.0	2.56	2.0	4.95	7.54
6	Brighton	11.75	14.89	10.51	1.0	2.29	3.17	1.0	1.46	1.71	2.0	3.75	4.69
7	Chelsea	12.4	16.02	12.51	1.0	2.35	3.67	1.0	1.81	2.3	2.0	4.15	5.74
8	Crystal Palace	19.4	18.79	13.15	1.0	2.18	3.57	1.0	1.73	2.29	2.0	3.91	5.14
9	Everton	17.6	18.9	10.66	1.0	1.77	2.51	1.0	1.23	1.48	2.0	3.0	2.86
10	Fulham	19.7	18.13	11.75	0.0	2.3	3.5	1.0	1.91	2.63	2.0	4.22	4.88
11	Ipswich Town	12.0	13.84	9.65	0.0	1.17	2.32	0.5	0.87	1.07	1.5	2.03	2.76
12	Leicester City	15.5	15.46	9.9	0.0	1.22	2.15	0.0	0.93	1.38	1.0	2.15	3.01
13	Liverpool	20.25	18.95	11.9	1.0	3.86	6.66	2.0	2.95	3.92	3.5	6.82	10.36
14	Manchester City	18.6	16.65	9.48	1.0	2.84	4.59	0.0	2.04	2.67	3.0	4.88	5.56
15	Manchester Utd	12.35	13.87	10.63	0.0	1.4	2.34	0.0	0.97	2.14	0.5	2.37	4.22
16	Newcastle Utd	19.5	18.13	12.56	0.0	2.87	5.21	1.0	2.17	2.96	2.0	5.04	7.4
17	Nott'ham Forest	21.55	18.98	13.38	1.0	2.59	4.45	1.0	1.91	2.88	1.5	4.5	6.25
18	Southampton	12.9	13.83	10.08	0.5	0.83	1.05	0.0	0.53	0.94	1.0	1.37	1.54
19	Tottenham	15.6	15.42	7.64	1.0	2.26	3.32	1.0	1.7	2.38	2.0	3.96	5.21
20	West Ham	12.9	16.67	10.87	0.0	1.72	3.16	1.0	1.16	1.84	1.0	2.88	4.68
21	Wolves	15.45	17.31	10.73	1.0	2.21	3.98	1.0	1.75	2.35	2.0	3.96	5.63

1	Metric	BestTeam
2	90s	Nott'ham Forest
3	Gls	Liverpool
4	Ast	Liverpool
5	G+A	Liverpool
6	G-PK	Liverpool
7	PK	Liverpool
8	PKatt	Liverpool
9	xG	Liverpool
10	npG	Liverpool

```

metric
90s      Nott'ham Forest
Gls      Liverpool
Ast      Liverpool
G+A      Liverpool
G-PK     Liverpool
...
Fls      Bournemouth
PKwon    Bournemouth
Recov    Crystal Palace
Won      Brentford
Won%     Brentford
Name: BestTeam, Length: 158, dtype: object

Đội có phong độ tổng thể cao nhất EPL 2024-2025:
Liverpool

```

3.2 đề xuất 1 phương pháp định giá cầu thủ.

3.2.1: Giới thiệu bài toán

Dựa vào kết quả từ câu I, đề xuất 1 phương pháp định giá cầu thủ.

3.2.2: Giải thích cách làm

3.2.2.1: Các thư viện cần dùng

- **sqlite3**: Thư viện chuẩn của Python dùng để kết nối và truy vấn SQLite database.
- **pandas**: Xử lý dữ liệu dạng bảng (DataFrame), lọc – chuyển đổi – xuất CSV.

- **numpy**: Hỗ trợ tính toán số học, xử lý giá trị thiếu (NaN).
- **os**: Tạo và xử lý đường dẫn đến file dữ liệu trong dự án.
- **matplotlib.pyplot**: Vẽ biểu đồ so sánh giá trị thực và giá trị dự đoán.
- **train_test_split** (sklearn): Chia dữ liệu thành tập huấn luyện và kiểm tra.
- **StandardScaler**: Chuẩn hóa dữ liệu số (đưa về cùng thang đo).
- **OneHotEncoder**: Mã hóa dữ liệu phân loại như vị trí, câu lạc bộ.
- **ColumnTransformer**: Xử lý từng nhóm cột bằng các phương pháp khác nhau.
- **Pipeline**: Gộp quy trình xử lý dữ liệu + huấn luyện vào một bước thống nhất.
- **RandomForestRegressor**: Mô hình học máy dùng để dự đoán giá trị cầu thủ.
- **mean_absolute_error, r2_score**: Đánh giá chất lượng mô hình dự đoán.

3.2.2.2. Giải thích từng hàm

1 – Kết nối database và đọc dữ liệu 2 bảng

```
BASE_DIR =
os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
DB_PATH = os.path.join(BASE_DIR, "data", "stats.db")
conn = sqlite3.connect(DB_PATH)
query = """
SELECT
    s.Player,
    s.Squad,
    s.Pos,
    s.Age,
    s.Min,
    s.Gls,
    s.Ast,
    s.xG,
    s.xAG,
    s.PrgP,
    s.Tkl,
    s.Int,
    s."Cmp%",
    v.transfer_value
FROM PLAYER_STATS s
LEFT JOIN PLAYER_VALUES v
ON s.Player = v.player_name
"""
df = pd.read_sql_query(query, conn)
conn.close()
```

- Kết nối vào file stats.db bằng sqlite3.
- Chạy câu lệnh SQL JOIN để lấy:
 - + Thống kê cầu thủ từ PLAYER_STATS
 - + Giá trị chuyển nhượng từ PLAYER_VALUES
- Đưa toàn bộ dữ liệu vào DataFrame df.
- Kiểm tra số dòng để chắc chắn dữ liệu đã tải thành công.

Ý nghĩa: Đây là bước nhập dữ liệu thô từ CSDL trước khi xử lý và huấn luyện mô hình.

2 – Xử lý giá chuyển nhượng

```
def parse_value(v):
    if pd.isna(v):
        return np.nan

    try:
        return float(v.replace("€", "").replace("M",
        "").strip())

    except:
        return np.nan

df["transfer_value_num"] =
df["transfer_value"].apply(parse_value)

df = df.dropna(subset=["transfer_value_num"])
```

- Chuyển giá trị dạng "€21.5M" → 21.5.
- Nếu dữ liệu hỏng hoặc "NA" → chuyển thành NaN.
- Loại bỏ các cầu thủ không có giá trị chuyển nhượng (không thể huấn luyện)

3 – Chuyển đổi dữ liệu số và điền thiếu

```
numeric_cols = ["Age", "Min", "Gls", "Ast", "xG", "xAG",
"PrgP", "Tkl", "Int", "Cmp%"]

for col in numeric_cols:
```

```
df[col] = pd.to_numeric(df[col], errors="coerce")
df[numeric_cols] = df[numeric_cols].fillna(0)
```

- Ép các cột thống kê về đúng kiểu float.
- Nếu có lỗi chuyển thành NaN.
- Điền toàn bộ giá trị thiếu bằng 0 để mô hình không bị lỗi.

Ý nghĩa: Làm sạch dữ liệu số trước khi xây dựng mô hình học máy.

4 – Chia dữ liệu thành đặc trưng (X) và nhãn (y)

```
features = numeric_cols + ["Pos", "Squad"]
X = df[features]
y = df["transfer_value_num"]
```

- X gồm các cột mô tả cầu thủ:
 - o số phút, bàn thắng, kiến tạo, xG, vị trí, đội bóng,...
- y là giá trị chuyển nhượng thực tế (M€).

Ý nghĩa: Tách dữ liệu đúng format để huấn luyện mô hình.

5 – Tạo pipeline tiền xử lý

```
preprocess = ColumnTransformer([
    ("num", StandardScaler(), num_cols),
    ("cat", OneHotEncoder(handle_unknown="ignore"), cat_cols)
])
```

- Chuẩn hóa dữ liệu số (mean = 0, std = 1).
- Mã hóa dữ liệu phân loại (Pos, Squad) thành dạng one-hot.
- Kết hợp trong ColumnTransformer để xử lý song song.

Ý nghĩa: Chuẩn hoá dữ liệu trước khi mô hình RandomForest tiếp nhận.

6 – Chia train/test

```
X_train, X_test, y_train, y_test = train_test_split(X, y,
    test_size=0.2, random_state=42)
```

- 80% dữ liệu dùng để train.

- 20% giữ lại để kiểm tra chất lượng mô hình.

Ý nghĩa: Đảm bảo đánh giá kết quả khách quan, không overfit.

7 – Huấn luyện mô hình

```
model = RandomForestRegressor(n_estimators=400, max_depth=10,
                              random_state=42)

pipeline = Pipeline([("preprocess", preprocess), ("model",
model)])

pipeline.fit(X_train, y_train)
```

- Dùng mô hình Random Forest để dự đoán giá trị cầu thủ.
- `n_estimators = 400`: dùng 400 cây quyết định.
- `max_depth = 10`: tránh mô hình học quá sâu.
- Pipeline xử lý dữ liệu + train mô hình trong cùng một bước.

Ý nghĩa: Cho phép dự đoán giá trị dựa trên thống kê chơi bóng.

8 – Đánh giá mô hình dự đoán

```
y_pred = pipeline.predict(X_test)

mae = mean_absolute_error(y_test, y_pred)

r2 = r2_score(y_test, y_pred)
```

- **MAE**: sai số dự đoán trung bình.
- **R²**: mô hình giải thích được bao nhiêu % biến thiên giá trị.

Ý nghĩa: Kiểm tra chất lượng mô hình trên dữ liệu thật.

9 – Dự đoán cho toàn bộ cầu thủ

```
df["Predicted_Value"] = pipeline.predict(X)

df["Difference"] = df["Predicted_Value"] -
df["transfer_value_num"]

df["Abs_Error"] = df["Difference"].abs()
```

- Tính giá trị dự đoán cho từng cầu thủ.
- So sánh với giá trị thật để xem sai lệch.

Ý nghĩa: Tạo bảng kết quả hoàn chỉnh để phân tích và báo cáo.

10 - Vẽ biểu đồ

```
plt.figure(figsize=(6, 6))

plt.scatter(df["transfer_value_num"], df["Predicted_Value"],
            alpha=0.7)

plt.plot([0, max(df["transfer_value_num"])], [0,
max(df["transfer_value_num"])], "r--")

plt.xlabel("Giá trị thực (€M)")

plt.ylabel("Giá trị dự đoán (€M)")

plt.title("So sánh Giá trị Thực tế vs Dự đoán")

plt.grid(True, alpha=0.3)

plt.tight_layout()

plt.show()
```

- Vẽ biểu đồ phân bố giữa giá trị thật và giá trị dự đoán.
- Đường đỏ là đường dự đoán hoàn hảo.

Ý nghĩa: Kiểm tra trực quan xem mô hình hoạt động tốt hay không.

3.2.2.3 Luồng xử lý của chương trình

1 – Kết nối database và lấy dữ liệu

- Mở file stats.db.
- JOIN hai bảng PLAYER_STATS và PLAYER_VALUES.
- Đọc toàn bộ vào DataFrame để chuẩn bị phân tích.

2 – Tiền xử lý dữ liệu

- Chuyển giá trị chuyển nhượng "€xxM" → số thực.
- Loại bỏ cầu thủ không có giá trị chuyển nhượng.
- Ép tất cả cột thống kê về dạng số và điền thiếu bằng 0.
- Chia dữ liệu thành:
 - + **Đặc trưng (X)**: thống kê + vị trí + đội bóng
 - + **Nhãn (y)**: giá trị chuyển nhượng thật

3 – Chia dữ liệu train / test

- 80% train – 20% test để kiểm tra mô hình.

4 – Xây dựng pipeline xử lý dữ liệu

- Chuẩn hóa cột số (StandardScaler).
- One-hot encode cho Pos và Squad.
- Gộp vào một pipeline cùng mô hình Random Forest.

5 – Huấn luyện mô hình RandomForest

- Train trên X_{train} , y_{train} .
- Dự đoán X_{test} và tính MAE, R^2 để đánh giá độ chính xác.

6 – Dự đoán cho toàn bộ cầu thủ

- Tính giá trị dự đoán cho từng cầu thủ.
- Tính chênh lệch và sai số tuyệt đối.
- Xuất kết quả ra file CSV predicted_values.csv.

7 – Tổng hợp kết quả

- Sai số trung bình toàn bộ dataset.
- Top cầu thủ bị định giá cao/thấp nhất so với thực tế.

8 – Vẽ biểu đồ

- Scatter plot giữa giá trị thật và giá trị dự đoán.
- Giúp quan sát sự khớp giữa mô hình và dữ liệu thực.

3.2.3: Kết quả

```
Dữ liệu sẵn sàng với 515 cầu thủ và 12 đặc trưng.
Số MAE: 0.90 M€
Hệ số R²: 0.646

Đã lưu kết quả sang: /home/taitu/Documents/GitHub/BTL-Python/CODE/data/raw/predicted_values

===== Tổng hợp =====
MAE trung bình: 5.80 M€
Chênh lệch trung bình: 0.39 M€

Top 5 cầu thủ được định giá cao hơn thực tế:


| Player           | Squad           | Pos   | transfer_value_num | Predicted_Value | Difference |
|------------------|-----------------|-------|--------------------|-----------------|------------|
| Mohamed Salah    | Liverpool       | FW    | 45.8               | 103.627018      | 57.827018  |
| Nicolás González | Manchester City | MF    | 0.6                | 30.179305       | 29.579305  |
| Marcus Tavernier | Bournemouth     | FW,MF | 24.8               | 51.978900       | 27.178900  |
| Yoane Wissa      | Brentford       | FW    | 25.5               | 49.377997       | 23.877997  |
| Sávio            | Manchester City | FW,MF | 1.2                | 24.758580       | 23.558580  |

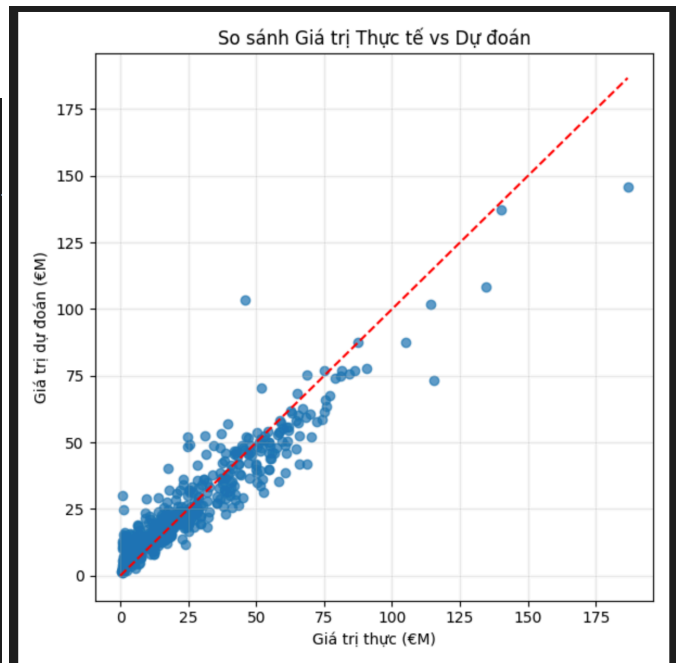


Top 5 cầu thủ được định giá thấp hơn thực tế:


| Player              | Squad           | Pos   | transfer_value_num | Predicted_Value | Difference |
|---------------------|-----------------|-------|--------------------|-----------------|------------|
| Cole Palmer         | Chelsea         | MF,FW | 115.4              | 73.455726       | -41.944274 |
| Erling Haaland      | Manchester City | FW    | 186.7              | 145.692790      | -41.007210 |
| Murillo             | Nott'ham Forest | DF    | 68.5               | 41.834348       | -26.665652 |
| Alexis Mac Allister | Liverpool       | MF    | 134.6              | 108.482218      | -26.117782 |
| Jeremy Monga        | Leicester City  | FW,MF | 66.0               | 42.086288       | -23.913712 |



Process finished with exit code 0
```



Câu 4:

4.1. Bài toán 1

Sử dụng thuật toán *K-means* để phân loại các cầu thủ thành các nhóm có chỉ số tương tự nhau. Theo bạn thì nên phân loại cầu thủ thành bao nhiêu nhóm? Vì sao (gợi ý: sử dụng biểu đồ *Elbow* và *Silhouette*, vẽ 2 biểu đồ này)? Bạn có Nhận xét gì về kết quả.

4.1.1. Các thư viện sử dụng

```
import sqlite3
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score
```

+ import sqlite3: Dùng để kết nối cơ sở dữ liệu SQLite.

+ import pandas as pd: xử lý dữ liệu dạng bảng.

- + import matplotlib.pyplot as plt: vẽ biểu đồ 2D
- + from sklearn.preprocessing import StandardScaler: chuẩn hóa dữ liệu (đưa các chỉ số về cùng thang đo).
- + from sklearn.cluster import Kmeans: Thuật toán K-Means Clustering – phân cụm dữ liệu thành k nhóm có đặc trưng tương tự.
- + from sklearn.metrics import silhouette_score: đánh giá chất lượng phân cụm.

4.1.2. Xử lý số liệu

```
# 2 Tiền xử lý dữ liệu
# Loại bỏ các cột không dùng cho phân cụm
drop_cols = [    'Player', 'Squad', 'Nation', 'Pos', 'Born', 'Matches',
                'Age', 'MP', 'Starts', 'Min', '90s',
                'Mn/MP', 'Min%', 'Mn/Start', 'Mn/Sub', 'Compl', 'Subs', 'unSub', 'PPM',
                'onG', 'onGA', '+/-', '+/-90', 'On-Off', 'On-Off.1',
                '2CrdY', 'FK', 'CK', 'OG', 'Err',

#loại bỏ các cột thể hiện các chỉ số phụ thuộc vào thời gian ra sân
                'Gls', 'Ast', 'G+A', 'G-PK', 'xG', 'npxG', 'xAG', 'npxG+xAG',
                'GA', 'SoTA', 'Saves', 'CS', 'PSxG',
                'Sh', 'SoT', 'TotDist', 'PrgDist', 'SCA', 'GCA', 'Tkl', 'Touches'
            ]
numeric_df = df.drop(columns=drop_cols, errors="ignore").copy()
```

- + Loại bỏ các cột các cột định danh, các cột về thời gian thi đấu và các chỉ số tích lũy theo thời gian thay vào đó sử dụng các cột per 90

```
# Chuyển toàn bộ cột về dạng số
numeric_df = numeric_df.apply(pd.to_numeric, errors='coerce')

# Điền toàn bộ giá trị N/A bằng 0
numeric_df = numeric_df.fillna(0)
print("Đã thay toàn bộ giá trị N/A bằng 0.\n")

# 3 Chuẩn hóa dữ liệu
scaler = StandardScaler()
X_scaled = scaler.fit_transform(numeric_df)
```

- + Chuyển các cột thành dạng số
- + Các chỉ số N/A chuyển về bằng 0

+ Chuẩn hóa dữ liệu về cùng 1 thang đo (các cột thống kê số liệu, các cột chỉ số phần trăm,...)

4.1.3. Sử dụng thuật toán K-Means để phân loại các cầu thủ thành các nhóm có chỉ số tương tự nhau

- Tóm tắt thuật toán:

- **Input:**

- K: số cluster
- Bộ dữ liệu: $\{x_1, x_2, \dots, x_n\}$ với $x \in \mathbb{R}^d$

- **Thuật toán:**

- 1. Khởi tạo ngẫu nhiên K tâm của các cụm (cluster centroids)
 $\mu_1, \mu_2, \dots, \mu_k$.
- 2. Lặp lại quá trình cập nhật tâm cụm cho tới khi dừng
 - Xác định nhãn cho từng điểm dữ liệu dựa vào khoảng cách tới tâm cụm
$$c_i = \arg \min_j \|x_i - \mu_j\|_2^2$$
 - Tính lại giá trị của từng tâm cụm

$$\mu_j := \frac{\sum_{i=1}^n \mathbf{1}(c_i = j) x_i}{\sum_{i=1}^n \mathbf{1}(c_i = j)}$$

```
inertias = []
sil_scores = []
K = range(2, 11)

for k in K:
    kmeans = KMeans(n_clusters=k, random_state=0, n_init=10)
    kmeans.fit(X_scaled)
    inertias.append(kmeans.inertia_)
    labels = kmeans.labels_
    sil_scores.append(silhouette_score(X_scaled, labels))
```

- Vòng lặp for k in K:

+ Vòng lặp chạy qua các giá trị số lượng cụm (K) từ 2 đến 10.

+ Mỗi lần lặp:

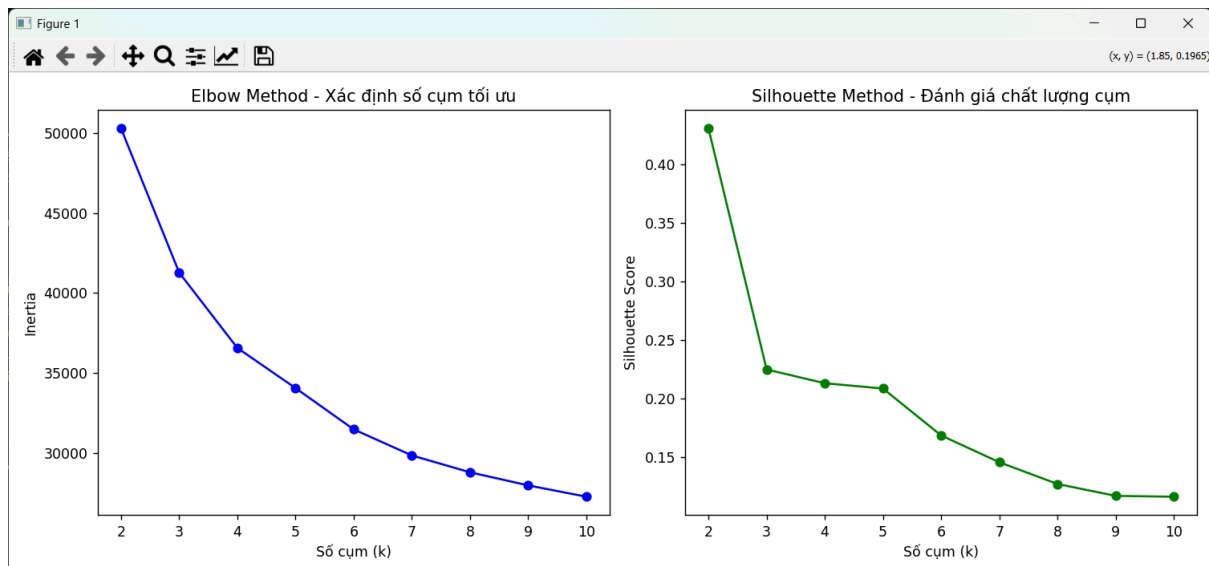
○ `kmeans = KMeans(n_clusters=k, random_state=0, n_init=10)`:

Khởi tạo mô hình K-Means với k cụm hiện tại.

- `random_state=0` đảm bảo kết quả có thể tái lập được (giống nhau mỗi lần chạy).
- `n_init=10` chỉ định thuật toán chạy 10 lần với các điểm khởi tạo khác nhau và chọn kết quả tốt nhất.

- `kmeans.fit(X_scaled)`: Huấn luyện mô hình K-Means trên tập dữ liệu đã được chuẩn hóa (`X_scaled`).
- Tính toán và lưu trữ chỉ số đánh giá:
 - + `inertias.append(kmeans.inertia_)` (Cho Elbow Method):
 - `kmeans.inertia_`: là giá trị Inertia (Tổng bình phương khoảng cách trong cụm - WCSS).
 - Giá trị này được lưu vào danh sách `inertias`. Mục tiêu là tìm K tại đó Inertia giảm chậm lại.
 - + `labels = kmeans.labels_`:
 - Thu thập nhãn cụm (cluster labels) mà mô hình K-Means đã gán cho từng điểm dữ liệu.
 - + `sil_scores.append(silhouette_score(X_scaled, labels))` (Cho Silhouette Analysis):
 - Sử dụng nhãn cụm vừa tìm được để tính Silhouette Score (Điểm Silhouette) của toàn bộ tập dữ liệu.
 - Điểm này được lưu vào danh sách `sil_scores`. Mục tiêu là tìm K cho ra điểm Silhouette cao nhất.

4.1.4. Vẽ biểu đồ Elbow và Silhouette



4.1.4.1. Elbow Method (Phương pháp Khuỷu tay)

- Biểu đồ bên trái (Inertia): Ta tìm điểm mà độ dốc của đường cong giảm chậm lại rõ rệt nhất, tạo thành "khuỷu tay".

- + Độ giảm của Inertia rất mạnh từ $K=2$ đến $K=4$.
- + Tại $K=4$, đường cong bắt đầu trở nên thoải hơn.
- + Tại $K=5$ hoặc $K=6$, độ giảm tiếp tục không quá rõ ràng.
- + Kết luận từ Elbow: Từ phương pháp này chọn $K=4$

4.1.4.2. Silhouette Method (Phương pháp Silhouette)

- Biểu đồ bên phải (Silhouette Score): Ta tìm giá trị K cho ra điểm Silhouette Score cao nhất. Điểm cao nhất (gần 1) cho thấy các cụm được phân tách rõ ràng và chặt chẽ nhất.
 - + Điểm cao nhất nằm tại $K=2$ (với Silhouette Score xấp xỉ 0.43).
 - + Từ $K=2$, điểm số giảm mạnh xuống $K=3$ (xấp xỉ 0.22) và tiếp tục giảm dần.
- Kết luận từ Silhouette: Từ phương pháp này chọn $K=2$

Kết luận:

- $K=4$ là điểm "khuỷu tay" rõ ràng trên biểu đồ Inertia, cho thấy việc tăng K lên 5, 6, 7 không mang lại lợi ích giảm lỗi đáng kể nữa.
- Mặc dù $K=2$ có Silhouette Score cao nhất, nhưng điểm Silhouette Score chung đều khá thấp (dưới 0.5), điều này ngụ ý rằng cấu trúc cụm của dữ liệu không quá mạnh mẽ, và việc tăng K từ 2 lên 4 giúp khai thác thêm thông tin chi tiết (4 nhóm thay vì 2) mà chỉ phải chấp nhận một sự giảm nhẹ không đáng kể về chất lượng cụm.

=>> Do đó nhóm em quyết định chọn $k=4$ và phân loại cầu thủ thành 4 nhóm vì:

- Bóng đá là môn thể thao đa dạng vai trò. Với 4 nhóm, việc phân loại sẽ chi tiết hơn so với 2 hoặc 3 nhóm, cho phép mô tả cầu thủ không chỉ ở các đặc trưng cơ bản (tấn công/phòng ngự) mà còn nhiều yếu tố khác như khả năng hỗ trợ, kiểm soát tuyến giữa hay tốc độ.
- Khi phân chia thành 4 nhóm, các cầu thủ thuộc cùng một nhóm sẽ có mức độ tương đồng về mặt chỉ số cao hơn. Điều này giúp tăng cường tính chính xác và ý nghĩa trong việc đánh giá đặc điểm, tiềm năng, và phong cách chơi đại diện cho cả nhóm.

Nhận xét: 4 nhóm được chia với các đặc điểm riêng khác biệt:

Số lượng cầu thủ trong mỗi cụm:

Cluster

0 250

2 136

3 75

1 41

Name: count, dtype: int64

- + Nhóm 1: bao gồm các cầu thủ có chỉ số đặc biệt các chỉ số liên quan trực tiếp đến việc bảo vệ khung thành và phát triển bóng từ phía sau. Thường nhóm này là thủ môn

252	Alisson	Liverpool	br BRA	GK	31	1
253	Alphonse Areola	West Ham	fr FRA	GK	31	1
254	Kepa Arrizabalaga	Bournemouth	es ESP	GK	29	1
255	Altay Bayindir	Manchester Utd	tr TUR	GK	26	1
256	Daniel Bentley	Wolves	eng ENG	GK	31	1
257	Martin Dúbravka	Newcastle Utd	sk SVK	GK	35	1
258	Ederson	Manchester City	br BRA	GK	30	1
259	Łukasz Fabiański	West Ham	pl POL	GK	39	1
260	Mark Flekken	Brentford	nl NED	GK	31	1
261	Fraser Forster	Tottenham	eng ENG	GK	36	1
262	Dean Henderson	Crystal Palace	eng ENG	GK	27	1
263	Mads Hermansen	Leicester City	dk DEN	GK	24	1
264	Sam Johnstone	Wolves	eng ENG	GK	31	1
265	Filip Jørgensen	Chelsea	dk DEN	GK	22	1
266	Caoimhín Kelleher	Liverpool	ie IRL	GK	25	1
267	Antonín Kinský	Tottenham	cz CZE	GK	21	1
268	Bernd Leno	Fulham	de GER	GK	32	1
269	Joe Lumley	Southampton	eng ENG	GK	29	1
270	Emiliano Martínez	Aston Villa	ar ARG	GK	31	1
271	Alex McCarthy	Southampton	eng ENG	GK	34	1
272	Arijanet Muric	Ipswich Town	xk KVM	GK	25	1
273	Neto	Bournemouth	br BRA	GK	35	1
274	Robin Olsen	Aston Villa	se SWE	GK	34	1
275	André Onana	Manchester Utd	cm CMR	GK	28	1
276	Stefan Ortega	Manchester City	de GER	GK	31	1
277	Alex Palmer	Ipswich Town	eng ENG	GK	27	1
278	Jordan Pickford	Everton	eng ENG	GK	30	1
279	Nick Pope	Newcastle Utd	eng ENG	GK	32	1

- + Nhóm 2: các đặc trưng về chỉ số phòng ngự cao, chỉ số tấn công thấp.
Thường nhóm này chủ yếu là hậu vệ, tiền vệ phòng ngự, trung vệ

395	Ethan Pinnock	Brentford	jm JAM	DF	31	2
396	Pedro Porro	Tottenham	es ESP	DF	24	2
397	Chris Richards	Crystal Palace	us USA	DF	24	2
398	Andrew Robertson	Liverpool	sct SCO	DF	30	2
399	Antonee Robinson	Fulham	us USA	DF	26	2
400	Cristian Romero	Tottenham	ar ARG	DF	26	2
401	William Saliba	Arsenal	fr FRA	DF	23	2
402	Fabian Schär	Newcastle Utd	ch SUI	DF	32	2
403	Nélson Semedo	Wolves	pt POR	DF	30	2
404	Bernardo Silva	Manchester City	pt POR	MF,FW	29	2
405	Tomáš Souček	West Ham	cz CZE	MF	29	2
406	Boubakary Soumaré	Leicester City	fr FRA	MF	25	2
407	Djed Spence	Tottenham	eng ENG	DF	23	2
408	Yukinari Sugawara	Southampton	jp JPN	DF,MF	24	2
409	James Tarkowski	Everton	eng ENG	DF	31	2
410	Kenny Tete	Fulham	nl NED	DF	28	2
411	Youri Tielemans	Aston Villa	be BEL	MF	27	2
412	Jurriën Timber	Arsenal	nl NED	DF	23	2
413	Jean-Clair Todibo	West Ham	fr FRA	DF	24	2
414	Sandro Tonali	Newcastle Utd	it ITA	MF	24	2
415	Pau Torres	Aston Villa	es ESP	DF	27	2
416	Kieran Trippier	Newcastle Utd	eng ENG	DF	33	2
417	Destiny Udogie	Tottenham	it ITA	DF	21	2
418	Manuel Ugarte	Manchester Utd	uy URU	MF	23	2
419	Virgil van Dijk	Liverpool	nl NED	DF	33	2
420	Joël Veltman	Brighton	nl NED	DF	32	2
421	Kyle Walker-Peters	Southampton	eng ENG	DF	27	2
422	Aaron Wan-Bissaka	West Ham	cd COD	DF	26	2

- + Nhóm 3: các đặc trưng về chỉ số tấn công cao: Goals, Assists, Shots, Successful Dribbles,.... . Thường nhóm này là tiền đạo, tiền vệ cánh, tiền vệ tấn công

476	Iliman Ndiaye	Everton	sn SEN	FW	24	3
477	Pedro Neto	Chelsea	pt POR	FW	24	3
478	Dango Ouattara	Bournemouth	bf BFA	FW	22	3
479	Cole Palmer	Chelsea	eng ENG	MF,FW	22	3
480	Andreas Pereira	Fulham	br BRA	MF	28	3
481	Declan Rice	Arsenal	eng ENG	MF	25	3
482	Morgan Rogers	Aston Villa	eng ENG	FW,MF	22	3
483	Georginio Rutter	Brighton	fr FRA	MF,FW	22	3
484	Bukayo Saka	Arsenal	eng ENG	FW,MF	22	3
485	Mohamed Salah	Liverpool	eg EGY	FW	32	3
486	Jadon Sancho	Chelsea	eng ENG	FW	24	3
487	Ismaila Sarr	Crystal Palace	sn SEN	MF	26	3
488	Sávio	Manchester City	br BRA	FW,MF	20	3
489	Kevin Schade	Brentford	de GER	FW	22	3
490	Antoine Semenyo	Bournemouth	gh GHA	FW	24	3
491	Emile Smith Rowe	Fulham	eng ENG	MF	24	3
492	Dominic Solanke	Tottenham	eng ENG	FW	26	3
493	Jørgen Strand Larsen	Wolves	no NOR	FW	24	3
494	Dominik Szoboszlai	Liverpool	hu HUN	MF	23	3
495	Marcus Tavernier	Bournemouth	eng ENG	FW,MF	25	3
496	Adama Traoré	Fulham	es ESP	FW,MF	28	3
497	Leandro Trossard	Arsenal	be BEL	FW	29	3
498	Jamie Vardy	Leicester City	eng ENG	FW	37	3
499	Ollie Watkins	Aston Villa	eng ENG	FW	28	3
500	Danny Welbeck	Brighton	eng ENG	FW	33	3
501	Yoane Wissa	Brentford	cd COD	FW	27	3
502	Chris Wood	Nott'ham Forest	nz NZL	FW	32	3
503	Martin Ødegaard	Arsenal	no NOR	MF	25	3

- + Nhóm 4: có đặc trưng là cân bằng giữa chỉ số tấn công và phòng ngự, tỉ lệ chuyền bóng cao. Nhóm này thường chơi ở vị trí giữa sân có xu hướng

chơi toàn diện phụ thuộc vào thể trận. Nhóm này thường là tiền vệ trung tâm, tiền vệ cánh, tiền vệ phòng ngự.

	Player	Squad	Nation	Pos	Age	Cluster	
1	Joshua Acheampong	Chelsea	eng ENG	DF		18	0
2	Tosin Adarabioyo	Chelsea	eng ENG	DF		26	0
3	Simon Adingra	Brighton	ci CIV	FW,MF		22	0
4	Emmanuel Agbadou	Wolves	ci CIV	DF		27	0
5	Kristoffer Ajer	Brentford	no NOR	DF		26	0
6	Nathan Aké	Manchester City	nl NED	DF		29	0
7	Carlos Alcaraz	Everton	ar ARG	FW,MF		21	0
8	Miguel Almirón	Newcastle Utd	py PAR	FW		30	0
9	Harry Amass	Manchester Utd	eng ENG	DF		17	0
10	Michail Antonio	West Ham	jm JAM	FW		34	0
11	Antony	Manchester Utd	br BRA	DF,MF		24	0
12	Julian Araujo	Bournemouth	mx MEX	DF		22	0
13	Cameron Archer	Southampton	eng ENG	FW		23	0
14	Adam Armstrong	Southampton	eng ENG	FW,MF		27	0
15	Marco Asensio	Aston Villa	es ESP	MF		28	0
16	Taiwo Awoniyi	Nott'ham Forest	ng NGA	FW		26	0
17	Jordan Ayew	Leicester City	gh GHA	FW,MF		32	0
18	Benoît Badiashile	Chelsea	fr FRA	DF		23	0
19	Leon Bailey	Aston Villa	jm JAM	FW,MF		26	0
20	Ross Barkley	Aston Villa	eng ENG	MF		30	0
21	Armel Bella Kotchap	Southampton	de GER	DF		22	0
22	Lucas Bergvall	Tottenham	se SWE	MF		18	0
23	Beto	Everton	gw GNB	FW		26	0
24	Philip Billing	Bournemouth	dk DEN	MF		28	0
25	Lamare Bogarde	Aston Villa	nl NED	DF,MF		20	0
26	Willy Boly	Nott'ham Forest	ci CIV	DF		33	0
27	Sven Botman	Newcastle Utd	nl NED	DF		24	0
28	Conor Bradlev	Liverpool	nir NIR	DF		21	0
29							

4.2. Bài toán 2

Sử dụng thuật toán PCA, giảm số chiều dữ liệu xuống 2 chiều, 3 chiều, vẽ hình scatter plot phân cụm các điểm dữ liệu trên mặt 2D, và khối 3D.

4.2.1. Thư viện cần thiết

```
import plotly.express as px
```

```
from sklearn.decomposition import PCA
```

```
import matplotlib.pyplot as plt
```

- plotly.express: Dùng để tạo biểu đồ tương tác (interactive), đặc biệt là biểu đồ phân cụm PCA 3D, cho phép người dùng di chuột để xem thông tin cầu thủ.

- PCA: Chứa thuật toán Phân tích thành phần chính (PCA), dùng để giảm số lượng chỉ số thống kê xuống còn 2 hoặc 3 chiều (PC1, PC2, PC3) để có thể trực quan hóa kết quả phân cụm trên mặt phẳng 2D/3D.

- matplotlib.pyplot: Dùng để vẽ các biểu đồ tĩnh cơ bản, như biểu đồ phân cụm PCA 2D.

4.2.1. Xác định k=4, huấn luyện kmeans

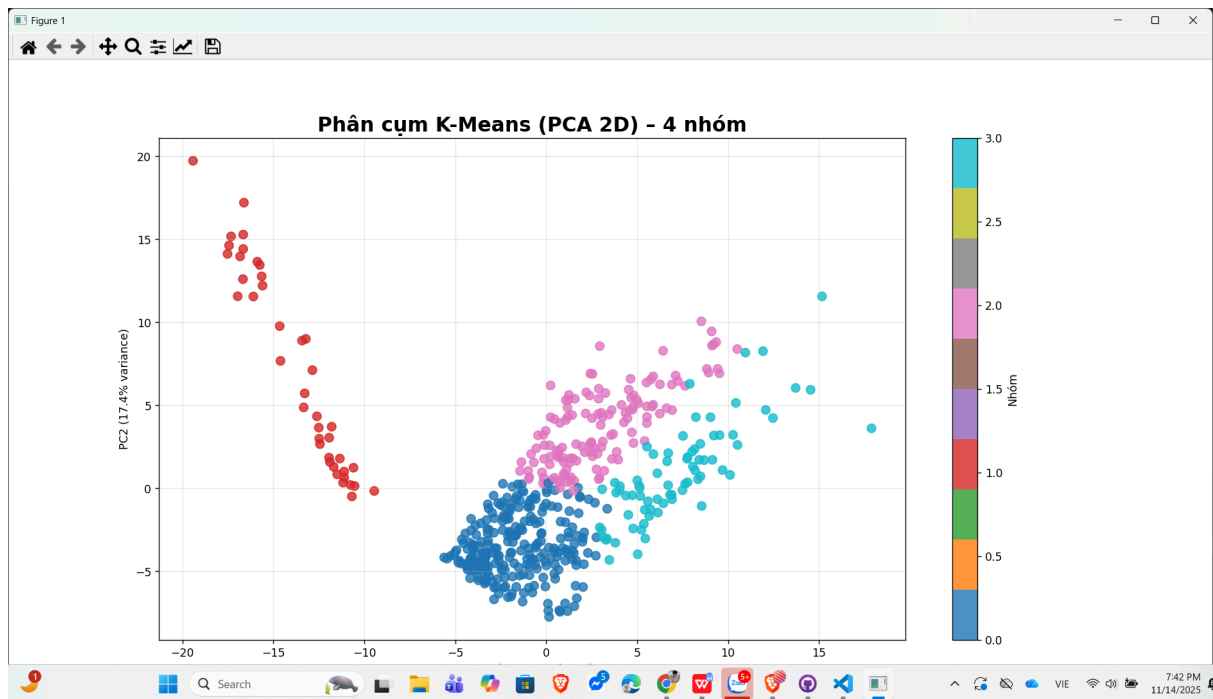
```
optimal_k = 4
print(f"\n Tiến hành phân cụm với k = {optimal_k}...\n")

kmeans = KMeans(n_clusters=optimal_k, random_state=0, n_init=40)
clusters = kmeans.fit_predict(X_scaled)
df['Cluster'] = clusters
df['Nhóm'] = df['Cluster']
print("Số lượng cầu thủ trong mỗi cụm:")
print(df["Cluster"].value_counts(), "\n")
```

4.2.2. Tạo mô hình PCA, giảm chiều xuống còn 2, 3 và vẽ biểu đồ

4.2.2.1. Giảm số chiều dữ liệu xuống thành 2 và vẽ biểu đồ 2D

```
pca2 = PCA(n_components=2)
X_2d = pca2.fit_transform(X_scaled)
plt.figure(figsize=(15,10))
scatter = plt.scatter(X_2d[:,0], X_2d[:,1], c=clusters, cmap='tab10', s=60, alpha=0.8)
plt.title('Phân cụm K-Means (PCA 2D) 4 nhóm', fontsize=18, fontweight='bold')
plt.xlabel(f'PC1 ({pca2.explained_variance_ratio_[0]:.1%} variance)')
plt.ylabel(f'PC2 ({pca2.explained_variance_ratio_[1]:.1%} variance)')
plt.colorbar(scatter, label='Nhóm')
plt.grid(True, alpha=0.3)
plt.show()
```



4.2.2.2. Giảm số chiều dữ liệu xuống thành 3 và vẽ biểu đồ 3D

```
pca3 = PCA(n_components=3)
X_3d = pca3.fit_transform(X_scaled)
data_3d = pd.DataFrame(X_3d, columns=['PC1', 'PC2', 'PC3'])
data_3d['Nhóm'] = df['Nhóm'].astype(str)
data_3d['Player'] = df['Player']
data_3d['Squad'] = df['Squad']
data_3d['Pos'] = df['Pos']
fig = px.scatter_3d(
    data_3d, x='PC1', y='PC2', z='PC3',
    color='Nhóm',
    hover_name='Player',
    hover_data=['Squad', 'Pos'],
    title='Phân cụm K-Means (PCA 3D) 4 nhóm',
    width=1000, height=700
)
fig.update_traces(marker=dict(size=4))
fig.show()
```